CODE FOR THIS TODO LIST:

```
#!/usr/bin/env python3
import tkinter as tk
from tkinter import simpledialog
import json
import os
import pathlib
from datetime import datetime

class TerminalWidget:
    def __init__(self, root):
        self.root = root
        self.root.title("SYS_TASK_SCHEDULER")

        # --- POSITIONING ---
        self.w, self.h = 350, 500
        screen_w = self.root.winfo_screenwidth()
        screen_h = self.root.winfo_screenheight()
        self.root.geometry(f"{self.w}x{self.h}+{screen_w - self.w - 20}+{screen_h - self.h - 60}")

        # --- THEME ---
        self.bg_color = "#0a0a0a"
        self.accent = "#00ff41"
        self.dim_green = "#003b00"
        self.root.configure(bg=self.bg_color)

        # --- STATE MANAGEMENT ---
        self.is_always_on_top = True  # Default state
        self.move_enabled = False
```

```
        self.root.attributes('-alpha', 0.92)
        self.root.attributes('-topmost', self.is_always_on_top)
        self.root.overrideredirect(True)

        # --- BINDINGS ---
        self.root.bind("<Control-t>", lambda e: self.toggle_overlap()) # Hotkey toggle
        self.root.bind("<Shift_L>", lambda e: self.set_move(True))
        self.root.bind("<KeyRelease-Shift_L>", lambda e: self.set_move(False))
        self.root.bind("<ButtonPress-1>", self.start_move)
        self.root.bind("<B1-Motion>", self.do_move)

        # --- DATA ---
        base_dir = pathlib.Path(os.environ.get('LOCALAPPDATA', pathlib.Path.home())) if os.name == 'nt'
else pathlib.Path.home() / ".local" / "share"
        app_data_dir = base_dir / 'AssaulterX'
        app_data_dir.mkdir(parents=True, exist_ok=True)
        self.file_path = app_data_dir / "terminal_tasks.json"

        self.tasks = self.load_tasks()
        self.setup_ui()
        self.check_daily_reset()
        self.blink_cursor()

    def toggle_overlap(self):
        """Toggles whether the window stays on top of others."""
        self.is_always_on_top = not self.is_always_on_top
        self.root.attributes('-topmost', self.is_always_on_top)
        self.update_status()

    def update_status(self):
        mode = "PINNED" if self.is_always_on_top else "FLOATING"
```

```
        lock = "MOVE_MODE" if self.move_enabled else "SYS_LOCKED"
        self.status_lbl.config(text=f"[ {mode} | {lock} ]")

    def set_move(self, state):
        self.move_enabled = state
        self.update_status()
        self.status_lbl.config(fg=self.accent if state else self.dim_green)

    def start_move(self, event):
        if self.move_enabled: self.x, self.y = event.x, event.y

    def do_move(self, event):
        if self.move_enabled:
            x = self.root.winfo_x() + (event.x - self.x)
            y = self.root.winfo_y() + (event.y - self.y)
            self.root.geometry(f"+{x}+{y}")

    def setup_ui(self):
        header = tk.Frame(self.root, bg=self.bg_color)
        header.pack(fill='x', pady=5)
        self.cursor_lbl = tk.Label(header, text=">_", font=("Courier", 12, "bold"), bg=self.bg_color,
fg=self.accent)
        self.cursor_lbl.pack(side="left", padx=5)
        tk.Label(header, text="ASSAULTER-X_v3.1", font=("Courier", 10), bg=self.bg_color,
fg=self.accent).pack(side="left")

        self.status_lbl = tk.Label(self.root, text="", font=("Courier", 8), bg=self.bg_color,
fg=self.dim_green)
        self.status_lbl.pack()
        self.update_status()
```

```
        self.entry = tk.Entry(self.root, font=("Consolas", 10), bg="#001100", fg=self.accent,
insertbackground=self.accent, relief="flat", borderwidth=5)
        self.entry.pack(pady=10, padx=15, fill='x')
        self.entry.bind('<Return>', lambda e: self.add_task())

        self.canvas = tk.Canvas(self.root, bg=self.bg_color, highlightthickness=0)
        self.frame = tk.Frame(self.canvas, bg=self.bg_color)
        self.canvas.pack(fill="both", expand=True, padx=10)
        self.canvas.create_window((0, 0), window=self.frame, anchor="nw", width=330)
        self.menu = tk.Menu(self.root, tearoff=0, bg=self.bg_color, fg=self.accent,
activebackground=self.dim_green)
        self.render_tasks()

    def load_tasks(self):
        if self.file_path.exists():
            with open(self.file_path, "r") as f: return json.load(f)
        return []

    def save_tasks(self):
        with open(self.file_path, "w") as f: json.dump(self.tasks, f, indent=4)

    def add_task(self):
        content = self.entry.get()
        if content:
            self.tasks.append({"content": content, "done": False, "reset_date":
datetime.now().strftime("%Y-%m-%d")})
            self.save_tasks(); self.entry.delete(0, tk.END); self.render_tasks()

    def render_tasks(self):
        for w in self.frame.winfo_children(): w.destroy()
        for i, t in enumerate(self.tasks):
            status = "[DONE]" if t['done'] else "[WAIT]"
```

```
        color = "#00d4ff" if t['done'] else self.accent
        lbl = tk.Label(self.frame, text=f"{status} {t['content']}", font=("Courier", 10), bg=self.bg_color, fg=color, anchor="w")
        lbl.pack(fill='x', pady=2)
        lbl.bind("<Button-1>", lambda e, idx=i: self.toggle_task(idx))
        lbl.bind("<Button-3>", lambda e, idx=i: self.show_menu(e, idx))

    def toggle_task(self, idx):
        self.tasks[idx]['done'] = not self.tasks[idx]['done']
        self.save_tasks(); self.render_tasks()

    def show_menu(self, event, idx):
        self.menu.delete(0, tk.END)
        overlap_label = "UNPIN (FLOAT)" if self.is_always_on_top else "PIN TO TOP"
        self.menu.add_command(label=f"➲ {overlap_label}", command=self.toggle_overlap)
        self.menu.add_separator()
        self.menu.add_command(label="✎ EDIT", command=lambda: self.edit_task(idx))
        self.menu.add_command(label="✕ DELETE", command=lambda: self.delete_task(idx))
        self.menu.add_separator()
        self.menu.add_command(label="EXIT SYSTEM", command=self.root.destroy)
        self.menu.post(event.x_root, event.y_root)

    def edit_task(self, idx):
        val = simpledialog.askstring("Terminal", "Update command:", initialvalue=self.tasks[idx]['content'])
        if val: self.tasks[idx]['content'] = val; self.save_tasks(); self.render_tasks()

    def delete_task(self, idx):
        self.tasks.pop(idx); self.save_tasks(); self.render_tasks()

    def blink_cursor(self):
```

```
        current = self.cursor_lbl.cget("text")
        self.cursor_lbl.config(text="> " if current == ">_" else ">_")
        self.root.after(500, self.blink_cursor)

    def check_daily_reset(self):
        today = datetime.now().strftime("%Y-%m-%d")
        for t in self.tasks:
            if t.get('reset_date') != today:
                t['done'] = False
                t['reset_date'] = today
        self.save_tasks(); self.render_tasks()
        self.root.after(60000, self.check_daily_reset)

if __name__ == "__main__":
    root = tk.Tk()
    app = TerminalWidget(root)
    root.mainloop()
```

SETUP STEPS FOR TODO LIST PYTHON APP FOR LOCAL DEVICES:

For Windows Users (The Executable)

1. **Install Python** and run: pip install pyinstaller
2. **Convert to EXE:** Run this command: pyinstaller --onefile --noconsole assaulter_x.py
3. **Auto-Startup:**
    - Press Win + R, type shell:startup.
    - Copy the .exe from the dist folder and **paste as shortcut** into the startup folder.

For Linux Users (The Script)

1. **Dependency:** sudo apt install python3-tk
2. **Permissions:** chmod +x assaulter_x.py
3. **Auto-Startup:** Create ~/.config/autostart/assaulter.desktop and add:

Control Interface Summary

- **Move Widget:** Hold **Left SHIFT** + Left-Click Drag.
- **Add Task:** Type in box and press **[ENTER]**.
- **Toggle Status: Left-Click** on any task.
- **Manage/Exit: Right-Click** on any task.
- **Toggle Overlap (Stay on Top):** Ctrl + T
- **Quick ToggleRight-click any task →** ➲ PIN/UNPIN